CampusHub – Smart Campus Facility, Booking & Inventory Management System

Complete Project Report · BCSE302P – Database Systems Lab · Vellore Institute of Technology, Vellore

Team member	Register number	Primary responsibility
Mohammed Affan	24BAI0014	Database design, schema, triggers and stored procedures
Hardik Gupta	24BCI0017	Flask backend, database connectivity and query/report module
Arham Sadiq K	24BAI0335	User interface, sample data, testing and documentation

Abstract

CampusHub is a centralized relational database system that manages campus facilities, facility bookings, equipment inventory, equipment issue/return and maintenance in one MySQL 8.0 database. The database has 10 normalized (3NF) tables linked by primary and foreign keys, and enforces its business rules inside the database through NOT NULL, UNIQUE, CHECK, DEFAULT and foreign key constraints, 5 triggers, 4 stored procedures and 4 views. Double booking is prevented by a trigger that rejects overlapping time slots and by a booking transaction that locks the facility row with `SELECT … FOR UPDATE`. A lightweight Python Flask web application sits on top of the database and gives students, faculty and administrators screens for searching facilities, booking slots, issuing and returning equipment, logging maintenance and viewing reports. Testing covered 20 functional test cases, including deliberate constraint violations and a two-session concurrency test, and all 20 behaved as expected.

Contents: 1 Introduction · 2 Requirement Analysis and Project Design · 3 Technology and Tools Used · 4 Database Implementation · 5 Application Implementation · 6 Testing and Results · 7 Challenges and Solutions · 8 Conclusion and Future Enhancements

Chapter 1: Introduction

1.1 Background

A university campus runs on shared resources: classrooms, computer labs, seminar halls, auditoriums and sports facilities, plus the projectors, microphones, laptops and sports kits kept inside them. At VIT these resources are used by thousands of students and hundreds of faculty members across many departments every day.

In most departments these resources are still managed through registers, spreadsheets, e-mail threads and phone calls. Each office keeps its own records, so nobody has a single, current view of which hall is free, which projector is out, or which lab has a pending repair. A database-driven system that keeps all of this information in one place, with the rules enforced by the database itself, solves this problem directly.

1.2 Problem Statement

The existing manual and disconnected way of managing campus facilities causes the following problems:

- **No real-time availability:** it is hard to know which facility is free for a given date and time slot.
- **Booking conflicts:** the same facility and slot can be booked twice, because two people can check a register at the same time.
- **Poor equipment tracking:** there is no reliable record of which equipment has been issued, to whom, and whether it has been returned.
- **No central inventory:** equipment records are kept in separate places by separate departments.

- **Incomplete maintenance history:** repairs are reported informally, so the history of a facility is lost.
- **Scattered records and weak reporting:** usage statistics such as the most-used halls or department-wise usage cannot be produced easily.

The project therefore needs a centralized relational database system for campus facilities, bookings, inventory and maintenance, in which data integrity and conflict-free booking are guaranteed by the database.

1.3 Objectives

1. Develop a centralized relational database for campus facilities.
2. Manage facility availability and bookings.
3. Maintain student, faculty, department and administrator information.
4. Track equipment inventory and available quantity.
5. Record equipment issue and return transactions.
6. Maintain facility maintenance records from report to completion.
7. Prevent booking conflicts using database constraints, triggers and transaction logic.
8. Generate useful reports using SQL queries and views.
9. Demonstrate DBMS concepts: ER modelling, normalization, constraints, indexing, views, triggers, stored procedures, transactions and concurrency control.

1.4 Scope

Included in the project	Excluded from the project
Student, faculty, department and admin management	Physical access-control hardware (smart locks, RFID gates)
Facility and facility-type management	Real payment gateway for paid facilities
Facility booking with conflict prevention	IoT sensors for live occupancy
Equipment inventory and issue/return	Full native mobile application
Maintenance management	Real-time building automation
Booking history and usage reports	Integration with the university ERP login
Database-level validation (constraints, triggers)	
A small web interface built with Flask	

CampusHub is primarily a DBMS project: the focus is data modelling, SQL, integrity and transactions. The Flask interface exists to demonstrate the database working end to end.

1.5 Motivation

All three of us have faced the problem first-hand: searching for a free room for a club event, finding a hall already taken despite an earlier verbal confirmation, or waiting for a projector that someone forgot to return. These problems are not caused by a lack of rooms; they are caused by a lack of reliable shared data.

The project was also an opportunity to apply every major topic of the Database Systems course to one realistic system. Booking conflicts are a natural, concrete example of why transactions, isolation and locking matter, and equipment stock is a natural example of why triggers are useful. Building CampusHub let us see these concepts working, rather than only reading about them.

Chapter 2: Requirement Analysis and Project Design

2.1 Users of the System

User	What they do in CampusHub
Student	Searches facilities, books and cancels slots, borrows and returns equipment, views own history

User	What they do in CampusHub
Faculty	Same as a student, plus booking larger halls for classes and events
Facility administrator	Manages facilities and equipment, issues and receives equipment, records and closes maintenance
Department administrator	Manages students and faculty of the department, views department-wise usage reports

2.2 Functional Requirements

ID	Requirement
FR1	Add, update, view and delete departments, students, faculty, admins, facility types and facilities
FR2	Search facilities by type, building and minimum capacity
FR3	Show the facilities that are free for a chosen date and time slot
FR4	Create a booking for a facility, date and time slot by a student or a faculty member
FR5	Reject any booking that overlaps an existing confirmed booking of the same facility
FR6	Reject bookings for a facility that is under maintenance or inactive
FR7	Cancel a booking (status changes to CANCELLED; the row is kept for history)
FR8	Maintain equipment inventory with total and available quantity per facility
FR9	Issue equipment to a student or faculty member only when a unit is available
FR10	Record the return of equipment and restore the available quantity automatically
FR11	Record a maintenance request; mark the facility as under maintenance until it is completed
FR12	Generate reports: facility usage, most-booked facilities, equipment availability, equipment currently issued, maintenance history, department-wise usage and monthly booking statistics

Non-functional requirements: data integrity enforced at the database level; no double booking even under concurrent requests; queries on bookings answer quickly as data grows (indexes); simple, browser-based interface; the schema is in 3NF to avoid update anomalies.

2.3 Data Requirements

Entity	Data stored
DEPARTMENT	Department ID, department name
STUDENT	Student ID, registration number, name, e-mail, phone, department
FACULTY	Faculty ID, employee code, name, e-mail, designation, department
ADMIN	Admin ID, name, e-mail, role (facility / department / super admin)
FACILITY_TYPE	Type ID, type name (classroom, computer lab, seminar hall, auditorium, sports)
FACILITY	Facility ID, name, type, managing admin, building, capacity, status
BOOKING	Booking ID, facility, booked by (student or faculty), date, start time, end time, purpose, status, created time
EQUIPMENT	Equipment ID, name, facility where it is kept, total quantity, available quantity, availability status
EQUIPMENT_ISSUE	Issue ID, equipment, issued to (student or faculty), issue date, due date, return date, status
MAINTENANCE	Maintenance ID, facility, handling admin, issue description, reported date, completion date, status

2.4 Database Selection

We selected **MySQL 8.0**, a relational database management system (RDBMS). CampusHub's data is highly structured and highly connected: every booking points to a facility and a person, every issue points to an equipment item and a person. This is exactly what the relational model handles well. MySQL was chosen over a NoSQL store for these reasons:

- **Relational model with keys:** primary and foreign keys enforce referential integrity between the 10 tables.

- **Constraints:** NOT NULL, UNIQUE, DEFAULT and, from MySQL 8.0.16, enforced CHECK constraints.
- **ACID transactions (InnoDB):** a booking is either fully recorded or not at all.
- **Row-level locking:** `SELECT … FOR UPDATE` lets us serialize concurrent bookings of the same facility.
- **Server-side logic:** triggers, stored procedures and views keep business rules in the database.
- **Joins and aggregation:** reports such as department-wise usage need multi-table joins with GROUP BY.
- **Practical reasons:** free and open source, used in our lab, and well supported by Python (mysql-connector-python) and MySQL Workbench.

2.5 Database Design

ER model and cardinality

The design has 10 entities: five master-data entities (DEPARTMENT, STUDENT, FACULTY, ADMIN, FACILITY_TYPE) and five operational entities (FACILITY, BOOKING, EQUIPMENT, EQUIPMENT_ISSUE, MAINTENANCE). All relationships are one-to-many.

Parent (1)	Relationship	Child (N)	Foreign key in child
DEPARTMENT	has	STUDENT	student.department_id
DEPARTMENT	employs	FACULTY	faculty.department_id
FACILITY_TYPE	classifies	FACILITY	facility.facility_type_id
ADMIN	manages	FACILITY	facility.admin_id
FACILITY	is reserved by	BOOKING	booking.facility_id
STUDENT / FACULTY	makes	BOOKING	booking.student_id / booking.faculty_id
FACILITY	houses	EQUIPMENT	equipment.facility_id
EQUIPMENT	is issued via	EQUIPMENT_ISSUE	equipment_issue.equipment_id
STUDENT / FACULTY	receives	EQUIPMENT_ISSUE	equipment_issue.student_id / faculty_id
FACILITY	has	MAINTENANCE	maintenance.facility_id
ADMIN	handles	MAINTENANCE	maintenance.admin_id

A booking or an equipment issue is made by **either** a student **or** a faculty member. Both foreign keys are therefore nullable, and a CHECK constraint guarantees that exactly one of them is filled. BOOKING and EQUIPMENT_ISSUE also act as the associative entities that resolve the many-to-many relationships between people and facilities, and between people and equipment.

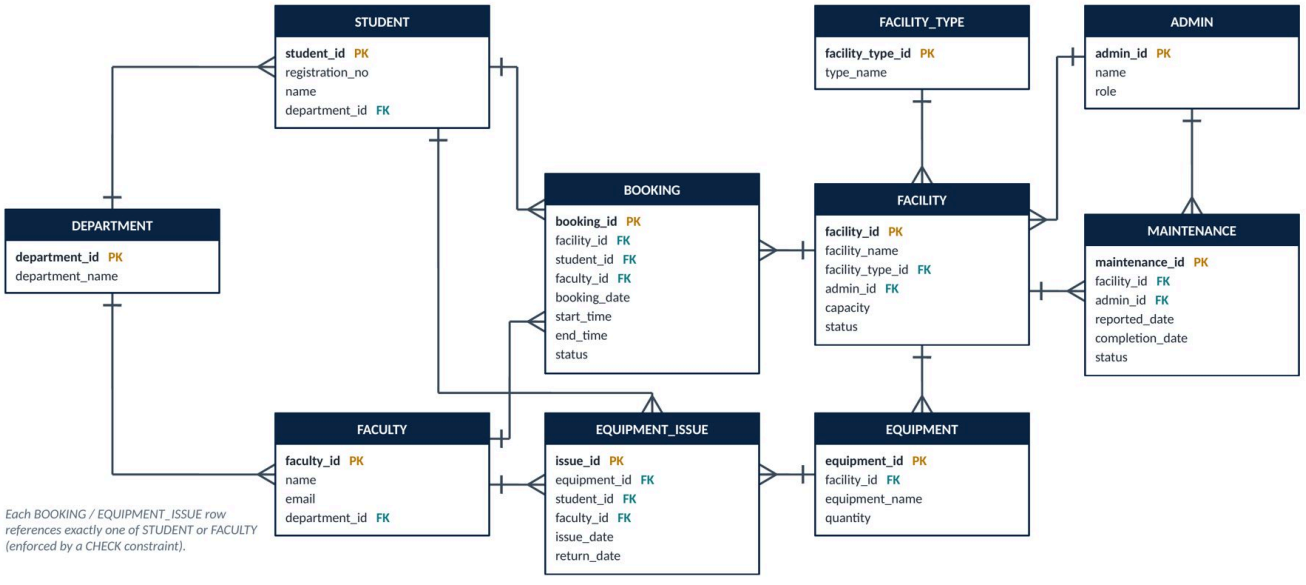


Figure 2: ER diagram of CampusHub (10 entities, all relationships 1 : N)

Relational schema (PK underlined in the ER diagram; FK marked)

```
DEPARTMENT(department_id PK, department_name UNIQUE)
FACILITY_TYPE(facility_type_id PK, type_name UNIQUE)
ADMIN(admin_id PK, name, email UNIQUE, role)
STUDENT(student_id PK, registration_no UNIQUE, name, email UNIQUE, phone, department_id FK)
FACULTY(faculty_id PK, employee_code UNIQUE, name, email UNIQUE, designation, department_id FK)
FACILITY(facility_id PK, facility_name UNIQUE, facility_type_id FK, admin_id FK, building, capacity,
status)
BOOKING(booking_id PK, facility_id FK, student_id FK, faculty_id FK, booking_date,
        start_time, end_time, purpose, status, created_at)
EQUIPMENT(equipment_id PK, facility_id FK, equipment_name, quantity, available_qty, availability_status)
EQUIPMENT_ISSUE(issue_id PK, equipment_id FK, student_id FK, faculty_id FK, issue_date,
        due_date, return_date, status)
MAINTENANCE(maintenance_id PK, facility_id FK, admin_id FK, issue_description,
        reported_date, completion_date, status)
```

Compared with the proposal, three small refinements were made during implementation: `available_qty` was added to EQUIPMENT so that stock can be tracked per unit, `due_date` was added to EQUIPMENT_ISSUE to find overdue items, and `purpose` and `created_at` were added to BOOKING for better history.

Normalization up to 3NF

We started from a single unnormalized booking record that stored student name, registration number, department name, facility name, facility type, building, capacity and a comma-separated list of equipment used.

- **1NF:** the multi-valued equipment field was removed; each equipment item became its own row in EQUIPMENT. Every column now holds one atomic value.
- **2NF:** student details depend only on the student and facility details only on the facility, not on the booking. They were moved to STUDENT and FACILITY, leaving BOOKING with only foreign keys and booking-specific attributes.
- **3NF:** department name depends on department_id, not directly on the student, and facility type name depends on facility_type_id. These transitive dependencies were removed into DEPARTMENT and FACILITY_TYPE.

The result: each fact is stored once. Changing a hall's capacity is one UPDATE on FACILITY, not one per booking, and there are no insert, update or delete anomalies. The only intentional derived value is `available_qty`, which is kept consistent by triggers (section 4.6).

2.6 Architecture

CampusHub follows a three-tier architecture: a browser-based presentation tier, a Flask application tier, and a MySQL data tier. Users never touch the database directly. The application sends parameterized SQL and stored-procedure calls through mysql-connector-python, and every data rule (constraints, triggers, transactions) is enforced in the MySQL tier, so the rules hold even if the database is used from another client.

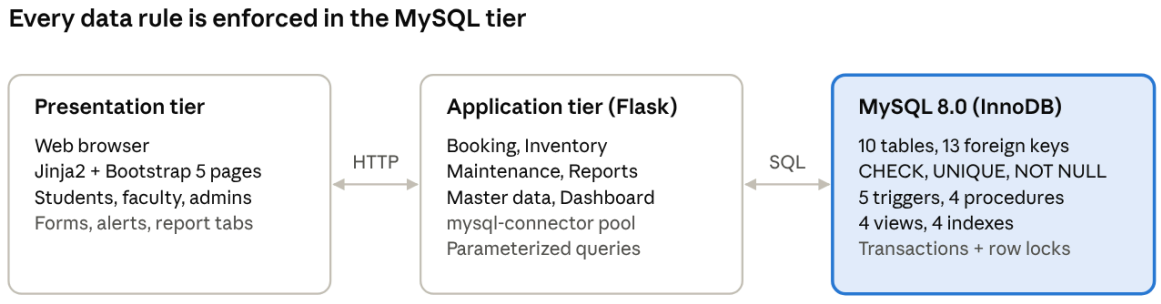


Figure 1: CampusHub three-tier architecture

Requests flow left to right: the browser posts a form, Flask turns it into a query or procedure call, and MySQL validates and stores it before the result flows back.

Chapter 3: Technology and Tools Used

Layer	Technology	Version	Why it was used
Database system	MySQL Community Server (InnoDB engine)	8.0	Relational model, enforced CHECK constraints, ACID transactions, row-level locks, triggers, procedures, views
Programming language	Python	3.11	Simple, readable backend language the team already knows
Web framework	Flask	3.x	Lightweight framework; routes map directly to database operations
Database driver	mysql-connector-python	8.x / 9.x	Official MySQL driver; supports parameterized queries, transactions and `callproc()` for stored procedures
Templates and UI	Jinja2 + HTML5 + Bootstrap 5	5.3	Server-rendered pages with responsive tables and forms, no separate frontend build
Database IDE	MySQL Workbench	8.0	Writing and running SQL scripts, viewing EER diagrams, checking query plans with EXPLAIN
Code editor	Visual Studio Code	latest	Editing Python, HTML and SQL files
Version control	Git and GitHub	–	Sharing code between the three team members
Environment config	python-dotenv	1.x	Keeps database host, user and password in a `.env` file instead of in code

3.1 Database System

MySQL 8.0 with the InnoDB storage engine stores all CampusHub data. InnoDB provides foreign keys, transactions with COMMIT and ROLLBACK, and row-level locking. Version 8.0.16 or later is required because older versions parse CHECK constraints but silently ignore them.

3.2 Programming Language and Framework

The application tier is written in Python using Flask. Flask was chosen over a heavier framework like Django because we wanted to write SQL ourselves rather than hide it behind an ORM; the purpose of the project is to demonstrate SQL, not to avoid it. Each Flask route runs one parameterized query or one stored-procedure call.

3.3 Development Environment

Development was done on our own laptops (Windows 11 and Ubuntu) with a local MySQL 8.0 server. The SQL is organised into four scripts that can rebuild the database from scratch in order: `01_schema.sql` (tables, constraints, indexes), `02_logic.sql` (triggers, procedures, views), `03_sample_data.sql` and `04_queries.sql` (report queries). The Flask app is started with `flask run` and opened at `http://127.0.0.1:5000`.

3.4 Cloud Services

No cloud service is required; the whole system runs locally. Because the application only needs a MySQL connection string, the same database could be moved to a managed service such as Amazon RDS for MySQL without code changes (see Future Enhancements).

Chapter 4: Database Implementation

The database `campushub` is built by two scripts: `01_schema.sql` creates the 10 tables, constraints and indexes, and `02_logic.sql` creates 5 triggers, 4 stored procedures and 4 views. All tables use the InnoDB engine and the utf8mb4 character set.

4.1 Database Creation

```
DROP DATABASE IF EXISTS campushub;
CREATE DATABASE campushub CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE campushub;
```

4.2 Table Creation (Data Structure Implementation)

Master tables are created first, then the tables that reference them. BOOKING is shown below as a representative definition; the other nine tables follow the same pattern (full script in the appendix files).

```
CREATE TABLE booking (
  booking_id   INT AUTO_INCREMENT PRIMARY KEY,
  facility_id  INT  NOT NULL,
  student_id   INT  NULL,
  faculty_id   INT  NULL,
  booking_date DATE NOT NULL,
  start_time   TIME NOT NULL,
  end_time     TIME NOT NULL,
  purpose      VARCHAR(200),
  status       ENUM('CONFIRMED','CANCELLED','COMPLETED') NOT NULL DEFAULT 'CONFIRMED',
  created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_booking_time   CHECK (end_time > start_time),
  CONSTRAINT chk_booking_person CHECK ((student_id IS NULL) <> (faculty_id IS NULL)),
  CONSTRAINT fk_booking_facility FOREIGN KEY (facility_id) REFERENCES facility(facility_id),
  CONSTRAINT fk_booking_student  FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE
RESTRICT,
  CONSTRAINT fk_booking_faculty  FOREIGN KEY (faculty_id) REFERENCES faculty(faculty_id) ON DELETE
RESTRICT
);
```

Data types were chosen to match the data: `DATE` and `TIME` for slots so that overlap checks are simple comparisons, `ENUM` for status columns so only valid states can be stored, and `VARCHAR` sized to realistic lengths (a VIT registration number fits in 15 characters).

4.3 Relationships and References

The 11 one-to-many relationships from section 2.5 are implemented as 13 foreign keys. Key decisions:

- **ON DELETE RESTRICT** on all parent references: a department, facility or person that still has bookings, issues or maintenance records cannot be deleted, so history is never orphaned. Cancelling is done by changing a status, not by deleting rows.
- **ON UPDATE CASCADE** on department references, so renumbering a department updates its students and faculty.
- **Either-or references:** BOOKING and EQUIPMENT_ISSUE each have two nullable foreign keys (`student_id`, `faculty_id`) with a CHECK that exactly one is set.

4.4 Constraints and Validation Rules

Constraint	Where	Rule enforced
PRIMARY KEY	every table	Unique, non-null row identifier (AUTO_INCREMENT)
UNIQUE	department_name, type_name, registration_no, employee_code, facility_name, all e-mails	No duplicate master records
NOT NULL	all mandatory columns	Required data must always be present
DEFAULT	facility.status = 'AVAILABLE', booking.status = 'CONFIRMED', created_at = CURRENT_TIMESTAMP	Sensible values when none is given
CHECK chk_capacity	facility	capacity > 0
CHECK chk_booking_time	booking	end_time > start_time
CHECK chk_booking_person	booking	exactly one of student_id / faculty_id is filled
CHECK chk_equipment_qty	equipment	0 ≤ available_qty ≤ quantity
CHECK chk_issue_dates	equipment_issue	due_date ≥ issue_date and return_date ≥ issue_date
CHECK chk_issue_return	equipment_issue	status is RETURNED exactly when return_date is set

Constraint	Where	Rule enforced
CHECK chk_maint_dates	maintenance	completion_date ≥ reported_date
CHECK chk_student_email	student	e-mail has the basic name@domain.tld shape
ENUM	all status and role columns	Only listed states are accepted
Trigger T1	booking	no overlapping confirmed booking; facility must be AVAILABLE
Trigger T2	equipment_issue	equipment can be issued only if a unit is available

Rules that compare a row with **other** rows (overlap, stock) cannot be written as CHECK constraints in MySQL, so they are enforced with triggers.

4.5 Indexes

```
CREATE INDEX idx_booking_slot ON booking (facility_id, booking_date, start_time);
CREATE INDEX idx_booking_date ON booking (booking_date);
CREATE INDEX idx_issue_status ON equipment_issue (status);
CREATE INDEX idx_facility_cap ON facility (capacity);
```

The composite index `idx_booking_slot` matches the overlap check exactly (facility, then date, then time), so the trigger and the availability search read only the bookings of one facility on one day. InnoDB also indexes every foreign key column automatically, and UNIQUE columns such as `registration_no` get a unique index.

4.6 Triggers

Trigger	Event	What it does
T1 trg_booking_before_insert	BEFORE INSERT on booking	Rejects the booking if the facility is not AVAILABLE or the slot overlaps a CONFIRMED booking
T2 trg_issue_before_insert	BEFORE INSERT on equipment_issue	Rejects the issue if available_qty is 0; otherwise decrements it and sets ALL_ISSUED when it reaches 0
T3 trg_issue_after_update	AFTER UPDATE on equipment_issue	On ISSUED → RETURNED, increments available_qty and sets status AVAILABLE
T4 trg_maint_after_insert	AFTER INSERT on maintenance	Sets the facility to UNDER_MAINTENANCE for any open request
T5 trg_maint_after_update	AFTER UPDATE on maintenance	When the last open request is completed, sets the facility back to AVAILABLE

```
CREATE TRIGGER trg_booking_before_insert
BEFORE INSERT ON booking
FOR EACH ROW
BEGIN
  IF (SELECT status FROM facility WHERE facility_id = NEW.facility_id) <> 'AVAILABLE' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Facility is under maintenance or inactive';
  END IF;
  IF NEW.status = 'CONFIRMED' AND EXISTS (
       SELECT 1 FROM booking b
       WHERE b.facility_id  = NEW.facility_id
         AND b.booking_date = NEW.booking_date
         AND b.status       = 'CONFIRMED'
         AND b.start_time   < NEW.end_time
         AND b.end_time     > NEW.start_time) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Time slot overlaps an existing booking';
  END IF;
END
```

Two time slots overlap when each one starts before the other ends; this single condition catches partial overlaps, exact matches and slots nested inside each other, while allowing back-to-back slots (10:00–12:00 then 12:00–13:00).

4.7 Stored Procedures and Transactions

Procedure	Purpose
sp_create_booking	Creates a booking inside a transaction, locking the facility row first

Procedure	Purpose
sp_issue_equipment	Issues one unit inside a transaction, locking the equipment row
sp_return_equipment	Marks an issue as RETURNED (trigger T3 restores the stock)
sp_record_maintenance	Logs a maintenance request (trigger T4 takes the facility out of service)

```
CREATE PROCEDURE sp_create_booking(
  IN p_facility_id INT, IN p_student_id INT, IN p_faculty_id INT,
  IN p_date DATE, IN p_start TIME, IN p_end TIME, IN p_purpose VARCHAR(200),
  OUT p_booking_id INT)
BEGIN
  DECLARE v_lock INT;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;
  START TRANSACTION;
    SELECT facility_id INTO v_lock FROM facility WHERE facility_id = p_facility_id FOR UPDATE;
    INSERT INTO booking (facility_id, student_id, faculty_id, booking_date, start_time, end_time, purpose)
    VALUES (p_facility_id, p_student_id, p_faculty_id, p_date, p_start, p_end, p_purpose);
    SET p_booking_id = LAST_INSERT_ID();
  COMMIT;
END
```

This is the transaction from the proposal implemented in SQL: BEGIN, check availability (trigger T1), create the booking, COMMIT; any error triggers the exit handler, which rolls back and passes the error message to the application. The `SELECT … FOR UPDATE` provides concurrency control: two sessions booking the same facility are forced to run one after the other (section 6.3).

4.8 Views

View	Report it provides
vw_facility_usage	Bookings and hours booked per facility (cancelled bookings excluded)
vw_equipment_availability	Total, available and issued quantity of each item, with its facility
vw_current_issues	Items currently issued, to whom, and whether they are overdue
vw_maintenance_history	All maintenance requests with facility, admin, status and days open

4.9 Sample Data

The script `03_sample_data.sql` loads realistic test data covering every table and every status value.

Table	Rows	Notes
department	5	CSE, AI & ML, ECE, Mechanical, Business School
facility_type	5	Classroom, Computer Lab, Seminar Hall, Auditorium, Sports Facility
admin	3	One of each role
student	8	Spread across all 5 departments
faculty	5	One per department
facility	8	Capacities from 20 to 600
booking	20	August to October 2026; 14 completed, 5 confirmed upcoming, 1 cancelled
equipment	8	43 units in total
equipment_issue	8	5 currently issued (2 of them overdue), 3 returned
maintenance	5	3 completed, 1 in progress, 1 reported

Bookings are loaded before maintenance rows, because trigger T1 refuses bookings for facilities that are already under maintenance. When the sample issues are loaded, trigger T2 lowers the available quantities automatically; when the open maintenance requests are loaded, trigger T4 switches the TT 105 Networks Lab and the Indoor Badminton Court to UNDER_MAINTENANCE.

Chapter 5: Application / System Implementation

The CampusHub web application is a Flask app of about 400 lines of Python plus 12 Jinja2 templates. It contains no business rules of its own: it collects input, calls a parameterized query or a stored procedure, and shows the result or the database's error message.

5.1 System Modules

Module	Flask routes	Main database objects used
Dashboard	`/`	COUNT queries on facility, booking, equipment_issue, maintenance
Master data	`/departments`, `/students`, `/faculty`, `/facilities`	INSERT / UPDATE / DELETE / SELECT on master tables
Facility search and availability	`/facilities/search`, `/availability`	facility, facility_type, booking (Q1, Q7)
Booking	`/bookings`, `/bookings/new`, `/bookings/<id>/cancel`	sp_create_booking, trigger T1, UPDATE booking
Inventory	`/equipment`, `/issues/new`, `/issues/<id>/return`	sp_issue_equipment, sp_return_equipment, triggers T2–T3
Maintenance	`/maintenance`, `/maintenance/new`, `/maintenance/<id>/complete`	sp_record_maintenance, triggers T4–T5
Reports	`/reports`	the 4 views and report queries Q3–Q10

5.2 User Interface

The interface is a set of server-rendered pages styled with Bootstrap 5, with a navigation bar linking the seven modules.

The main screens are a **dashboard** (summary cards and upcoming bookings), an **availability search** (date, slot, type and capacity filters with a *Book* button), a **booking form** with success/error alerts, an **inventory page** with badges for ALL_ISSUED and overdue items, a **maintenance page** with a *Mark completed* button, and a **reports page** with one tab per report.

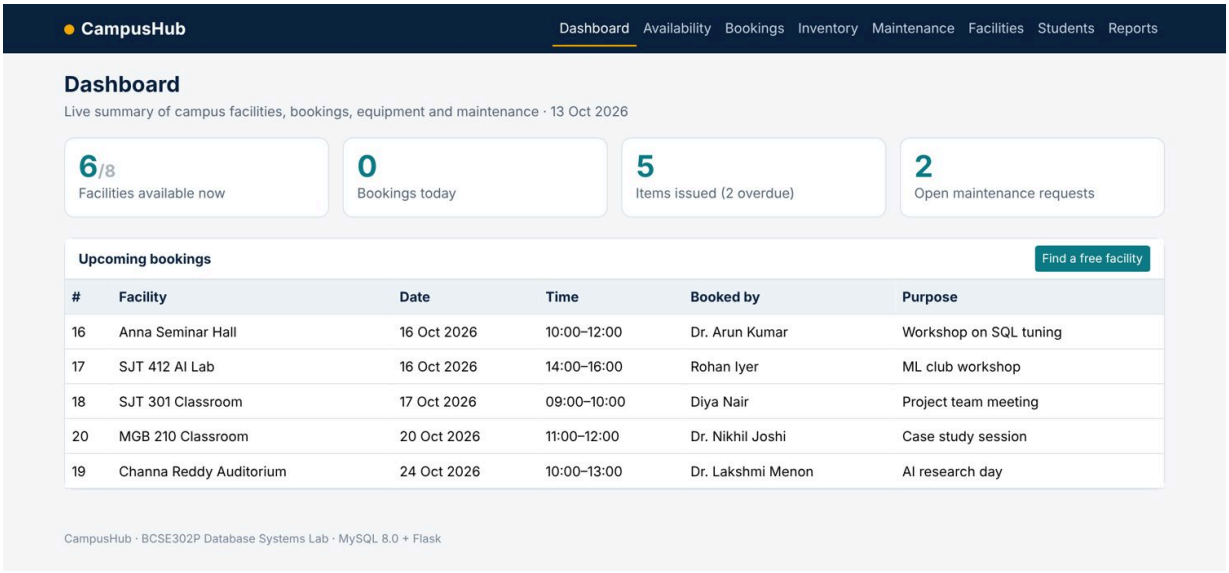


Figure 3: Dashboard – live summary cards and upcoming bookings

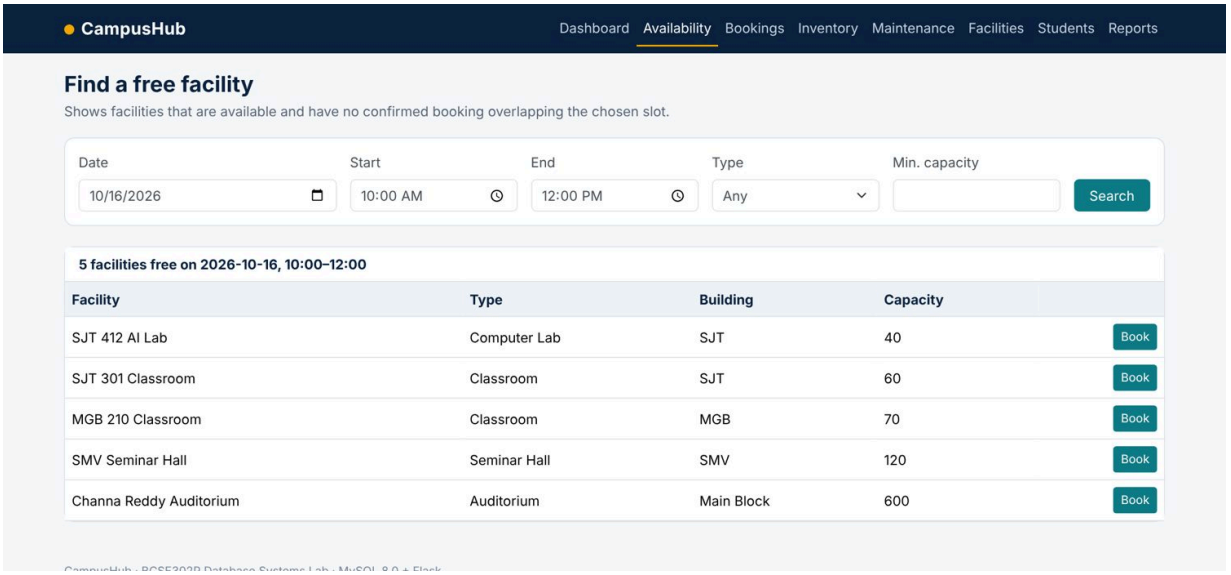


Figure 4: Availability search – 5 facilities free on 16 Oct 2026, 10:00–12:00

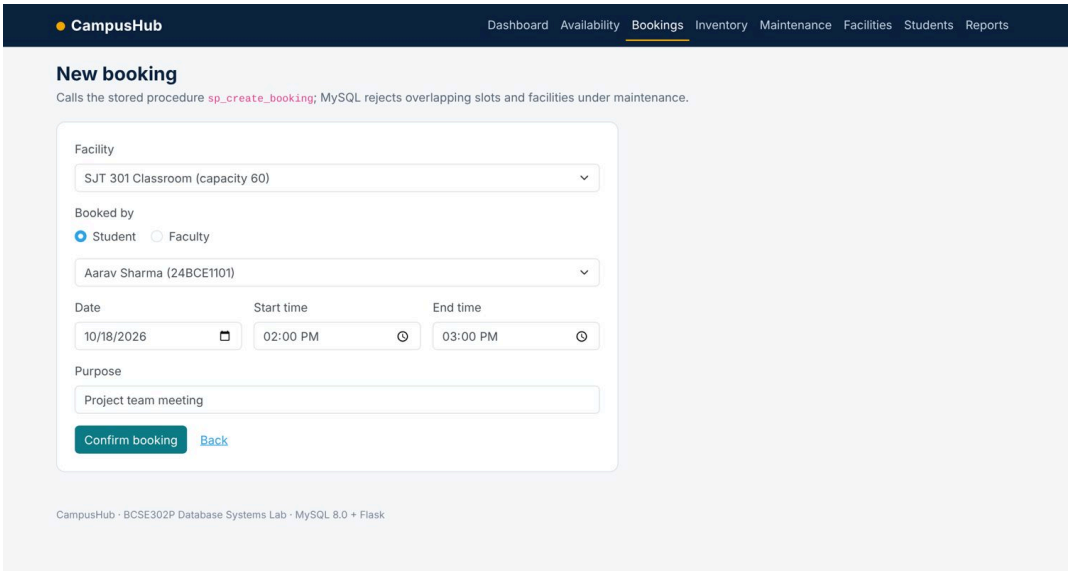


Figure 5: New booking form (calls sp_create_booking)

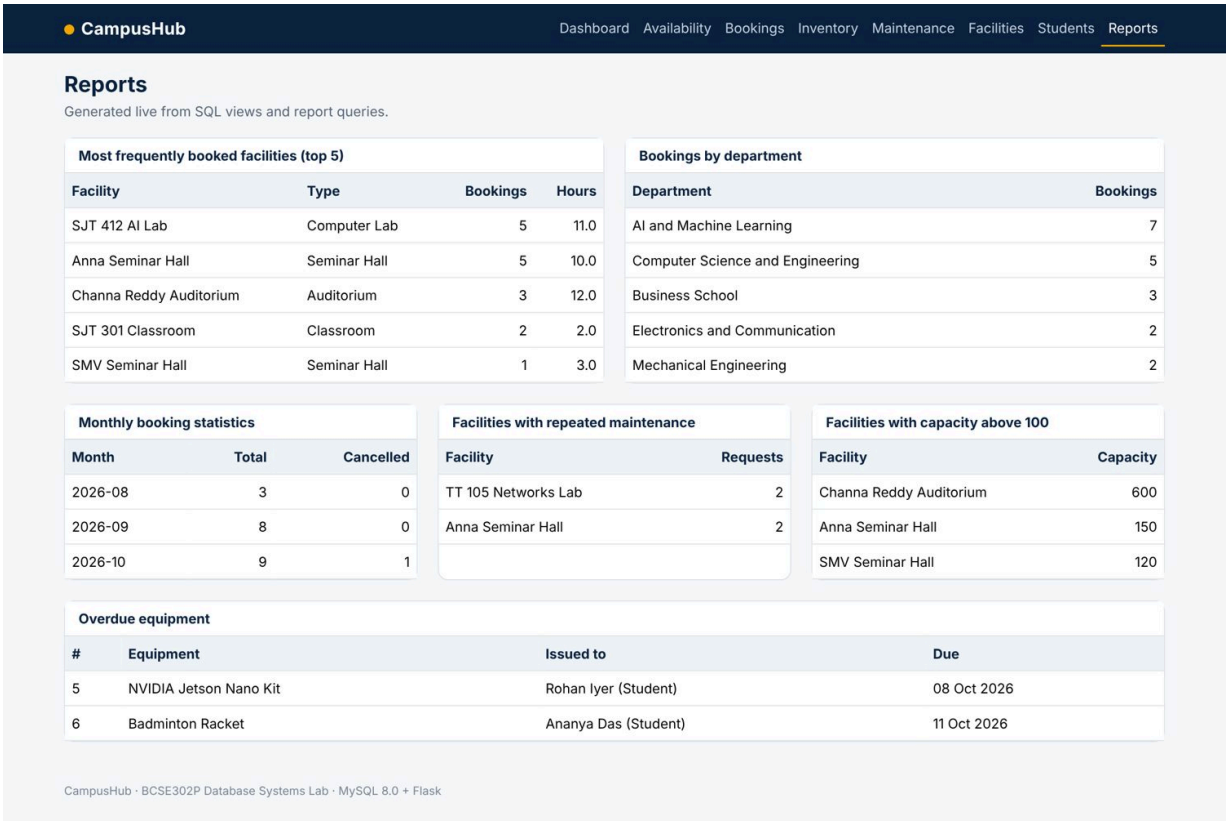


Figure 6: Reports page generated from SQL views and report queries

5.3 Database Connectivity

The application connects through mysql-connector-python using a small connection pool. Credentials are read from a `.env` file, never hard-coded.

```
pool = pooling.MySQLConnectionPool(pool_name="campushub", pool_size=5,
    host=os.getenv("DB_HOST"), user=os.getenv("DB_USER"),
    password=os.getenv("DB_PASSWORD"), database="campushub")
```

Every value from a form is passed as a parameter (`%s`), never concatenated into the SQL string, which protects the system from SQL injection.

5.4 Important Functionalities and Database Operations

The booking route shows how the application relies on the database for validation. It calls the stored procedure and turns any trigger or constraint error into a user-friendly message:

```
@app.post("/bookings/new")
def create_booking():
    f = request.form
    student = f.get("student_id") or None
    faculty = f.get("faculty_id") or None
    conn = pool.get_connection()
    try:
        cur = conn.cursor()
        args = cur.callproc("sp_create_booking",
            (f["facility_id"], student, faculty, f["date"], f["start"], f["end"], f["purpose"], 0))
        flash(f"Booking #{args[7]} confirmed.", "success")
    except Error as e:
        flash(e.msg, "danger")   # e.g. 'Time slot overlaps an existing booking'
    finally:
        conn.close()
    return redirect(url_for("bookings"))
```

Operation	SQL performed
Add / edit / delete student, faculty, facility	INSERT, UPDATE, DELETE (delete blocked by RESTRICT if history exists)
Book a facility	CALL sp_create_booking → trigger T1 → COMMIT or ROLLBACK
Cancel a booking	UPDATE booking SET status = 'CANCELLED' WHERE booking_id = %s
Issue equipment	CALL sp_issue_equipment → trigger T2 lowers stock

Operation	SQL performed
Return equipment	CALL sp_return_equipment → trigger T3 restores stock
Report maintenance	CALL sp_record_maintenance → trigger T4 closes the facility
Complete maintenance	UPDATE maintenance SET status = 'COMPLETED', completion_date = CURDATE() → trigger T5 reopens it

5.5 Data Management Workflow

1. An administrator sets up master data: departments, facility types, admins, students, faculty, facilities and equipment.
2. A student or faculty member searches for a free facility for a date and slot (query Q1).
3. They submit a booking; `sp_create_booking` locks the facility, trigger T1 validates the slot, and the booking is committed or rolled back.
4. Equipment needed for the event is issued with `sp_issue_equipment`; stock drops automatically. On return, `sp_return_equipment` restores it.
5. When a fault is reported, `sp_record_maintenance` logs it and the facility becomes unbookable until the request is completed.
6. After the event, bookings are marked COMPLETED; all history stays in the database and feeds the reports.

5.6 Queries and Searches

Query	Purpose	SQL features used
Q1	Free facilities for a slot	NOT EXISTS correlated subquery
Q2	All bookings of a facility	LEFT JOIN, COALESCE
Q3	Most-booked facilities	View, ORDER BY, LIMIT
Q4	Equipment currently issued	View with 4-table join
Q5	Facilities with repeated maintenance	GROUP BY, HAVING
Q6	Department-wise usage	LEFT JOIN on either-or keys, COALESCE in join
Q7	Facilities above a capacity	WHERE on indexed column
Q8	Monthly booking statistics	DATE_FORMAT, conditional SUM
Q9	Equipment availability	View
Q10	Maintenance history	View, DATEDIFF

5.7 Reports and Analytics

The Reports page reads directly from the four views, so every report is always current with no extra code. It shows facility usage (bookings and hours), the top 5 facilities, department-wise usage, monthly statistics, equipment availability, currently issued and overdue items, and maintenance history. Results for the sample data are in Chapter 6.

Chapter 6: Testing and Results

All 20 functional test cases passed: every valid operation succeeded, and every invalid one was rejected by the intended constraint, trigger or transaction with a clear error message. Results below were taken on the sample data, with 13 October 2026 as the current date.

6.1 Functional Test Cases

TC	Test	Input	Expected result	Status
1	Valid booking	SJT 301, 18 Oct, 14:00–15:00, student 1	Booking inserted, status CONFIRMED	Pass
2	Overlapping booking	Anna Seminar Hall, 16 Oct, 11:00–13:00	Rejected by T1: 'Time slot overlaps an existing booking'	Pass
3	Back-to-back booking	Anna Seminar Hall, 16 Oct, 12:00–13:00	Accepted (touching slots do not overlap)	Pass

TC	Test	Input	Expected result	Status
4	End time before start	15:00–14:00	Rejected by chk_booking_time (error 3819)	Pass
5	Both student and faculty set	student 1 and faculty 1	Rejected by chk_booking_person	Pass
6	Neither student nor faculty	both NULL	Rejected by chk_booking_person	Pass
7	Book a facility under maintenance	Indoor Badminton Court	Rejected by T1: 'Facility is under maintenance or inactive'	Pass
8	Non-existent facility	facility_id 99	Rejected by foreign key (error 1452)	Pass
9	Duplicate registration number	24BCE1101 again	Rejected by UNIQUE (error 1062)	Pass
10	Invalid capacity	capacity 0	Rejected by chk_capacity	Pass
11	Issue item with no stock	Document Camera (0 available)	Rejected by T2: 'No unit of this equipment is available'	Pass
12	Issue item with stock	Jetson Nano Kit	Issued; available_qty 3 → 2	Pass
13	Return equipment	issue 5	Status RETURNED; available_qty restored by T3	Pass
14	Return twice	issue 1 (already returned)	Rejected by sp_return_equipment	Pass
15	Report maintenance	SJT 301 Classroom	Request logged; facility becomes UNDER_MAINTENANCE (T4)	Pass
16	Complete maintenance	request from TC15	Facility becomes AVAILABLE again (T5)	Pass
17	Delete student with history	student 1	Rejected by ON DELETE RESTRICT (error 1451)	Pass
18	Cancel then rebook	cancel booking 18, rebook same slot	Second booking accepted (cancelled rows ignored)	Pass
19	Concurrent booking	two sessions, same facility and slot	Second session waits, then is rejected; one booking only	Pass
20	Atomic rollback	failing call to sp_create_booking	No partial row left; booking count unchanged	Pass

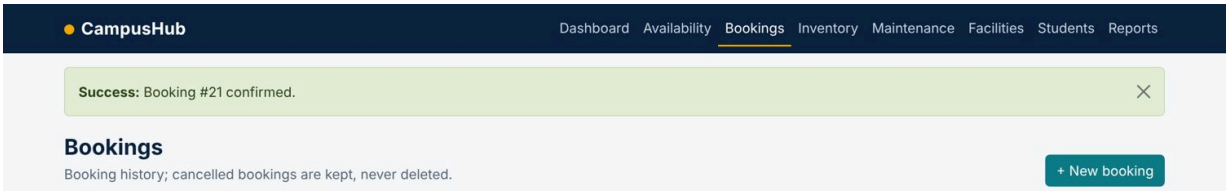


Figure 7: TC1 – valid booking accepted



Figure 8: TC2 – overlapping booking rejected by trigger T1

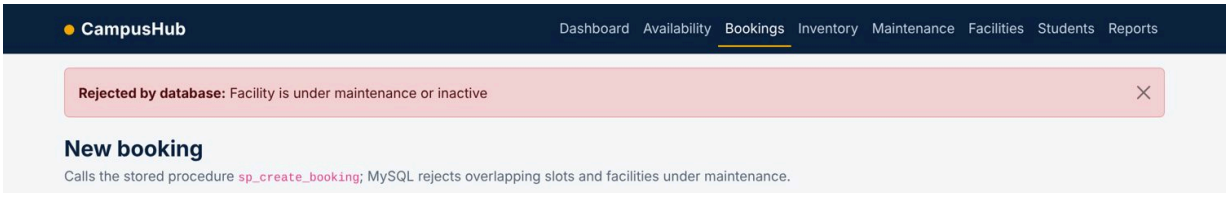


Figure 9: TC7 – booking a facility under maintenance rejected by trigger T1

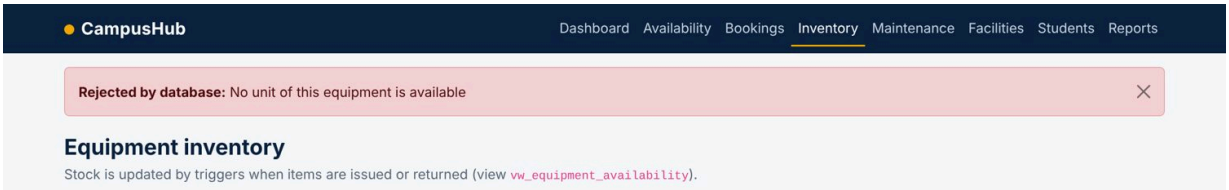


Figure 10: TC11 – issuing equipment with no stock rejected by trigger T2

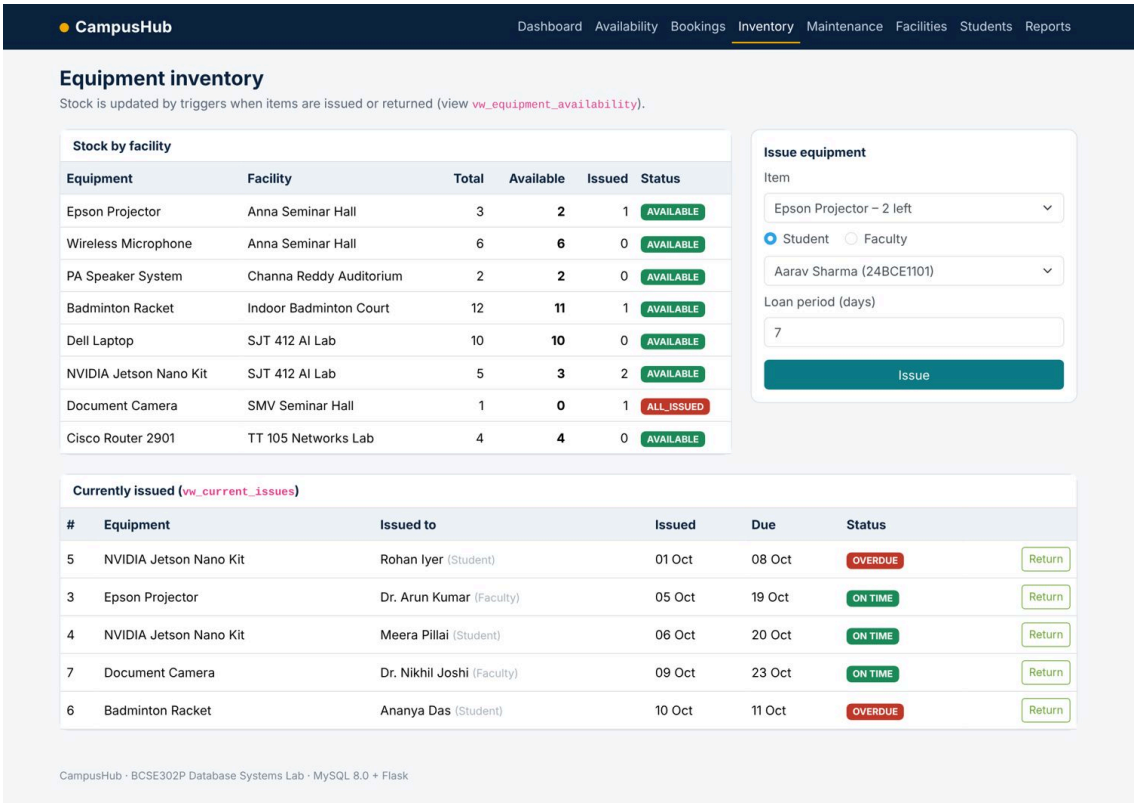


Figure 11: Inventory page – stock maintained by triggers, overdue items flagged by vw_current_issues

6.2 Output of Reports

Q1 – Facilities free on 16 Oct 2026, 10:00–12:00. Five of eight facilities are free. Anna Seminar Hall is excluded because it is already booked, and the Networks Lab and Badminton Court are excluded because they are under maintenance.

facility_name	type	building	capacity
SJT 412 AI Lab	Computer Lab	SJT	40
SJT 301 Classroom	Classroom	SJT	60
MGB 210 Classroom	Classroom	MGB	70
SMV Seminar Hall	Seminar Hall	SMV	120
Channa Reddy Auditorium	Auditorium	Main Block	600

Q3 – Most frequently booked facilities (top 5, cancelled excluded).

facility_name	total_bookings	hours_booked
SJT 412 AI Lab	5	11.0
Anna Seminar Hall	5	10.0
Channa Reddy Auditorium	3	12.0
SJT 301 Classroom	2	2.0
SMV Seminar Hall	1	3.0

Q4 – Equipment currently issued (vw_current_issues). Two items are overdue.

issue_id	equipment	issued_to	type	issue_date	due_date	due_status
5	NVIDIA Jetson Nano Kit	Rohan Iyer	Student	2026-10-01	2026-10-08	OVERDUE

issue_id	equipment	issued_to	type	issue_date	due_date	due_status
3	Epson Projector	Dr. Arun Kumar	Faculty	2026-10-05	2026-10-19	ON TIME
4	NVIDIA Jetson Nano Kit	Meera Pillai	Student	2026-10-06	2026-10-20	ON TIME
7	Document Camera	Dr. Nikhil Joshi	Faculty	2026-10-09	2026-10-23	ON TIME
6	Badminton Racket	Ananya Das	Student	2026-10-10	2026-10-11	OVERDUE

Q5 – Facilities with more than one maintenance record: TT 105 Networks Lab (2) and Anna Seminar Hall (2).

Q6 – Bookings by department (cancelled excluded).

department_name	bookings
AI and Machine Learning	7
Computer Science and Engineering	5
Business School	3
Mechanical Engineering	2
Electronics and Communication	2

Q7 – Facilities with capacity above 100: Channa Reddy Auditorium (600), Anna Seminar Hall (150), SMV Seminar Hall (120).

Q8 – Monthly booking statistics.

month	total bookings	cancelled
2026-08	3	0
2026-09	8	0
2026-10	9	1

Q9 – Equipment availability: stock values reflect trigger T2 running on the sample issues. The Document Camera shows 0 of 1 available (ALL_ISSUED), the Jetson Nano Kit 3 of 5, the Epson Projector 2 of 3 and the Badminton Racket 11 of 12; the other four items are fully available.

Q10 – Maintenance history: 5 requests, of which 3 are completed (open 2, 3 and 6 days), 1 is in progress (Indoor Badminton Court, 2 days) and 1 is reported (TT 105 Networks Lab, 1 day).

6.3 Concurrency Test (TC19)

Two MySQL Workbench sessions called `sp_create_booking` for SJT 301 Classroom on 18 Oct, 16:00–17:00, at the same moment.

Time	Session A	Session B
t1	START TRANSACTION; locks facility 1 (FOR UPDATE)	START TRANSACTION; requests lock on facility 1 → waits
t2	INSERT passes trigger T1	still waiting
t3	COMMIT; lock released	lock granted; INSERT checked by T1 → overlap found
t4	–	Exit handler: ROLLBACK, error 'Time slot overlaps an existing booking'

Exactly one booking was stored. Without the `FOR UPDATE` lock, both sessions can pass the overlap check before either commits, because under REPEATABLE READ neither sees the other's uncommitted row; the lock turns the two transactions into a serial schedule (A, then B), as planned in the proposal.

6.4 Performance Observations

- `EXPLAIN` on the overlap check and on Q1 shows MySQL using `idx_booking_slot` (access type `ref`), so only the bookings of one facility on one day are examined instead of the whole table.
- Q7 uses `idx_facility_cap` for the range condition on capacity.
- Joins on foreign keys use the automatic InnoDB foreign-key indexes, so all report queries return instantly on the sample data.

- The lock in `sp_create_booking` is on a single facility row, so bookings of **different** facilities still run fully in parallel; only competing requests for the same facility wait.
- Views add no storage cost; they are re-evaluated on each read, which is acceptable at campus scale.

Chapter 7: Challenges and Solutions

#	Type	Challenge	Solution implemented
1	Database design	A booking or equipment issue can belong to a student **or** a faculty member. A single `user_id` column cannot reference two tables.	Two nullable foreign keys plus `CHECK ((student_id IS NULL) <> (faculty_id IS NULL))`, so exactly one is always set and both remain real foreign keys.
2	Technical	Our first version used `ON DELETE SET NULL` on these foreign keys; MySQL refused to create the table (error 3823), because a column with a CHECK constraint cannot be changed by a referential action.	Switched to `ON DELETE RESTRICT`. This also matches the requirement to keep history: people with bookings are deactivated, not deleted.
3	Database design	Preventing double booking: the overlap rule compares a new row with **other** rows, which a CHECK constraint cannot do, and a UNIQUE key on (facility, date, start) misses partial overlaps such as 10:00–12:00 vs 11:00–13:00.	BEFORE INSERT trigger T1 with the overlap test `start < new_end AND end > new_start`, which catches every overlap and still allows back-to-back slots.
4	Technical (concurrency)	During testing, two sessions could both pass the trigger check before either committed, because under REPEATABLE READ neither sees the other's uncommitted row.	`sp_create_booking` locks the facility row with `SELECT … FOR UPDATE` before inserting, so competing bookings of the same facility are serialized (TC19).
5	Data handling	Keeping equipment stock correct: updating `available_qty` from the Flask code risked forgetting it on some path, or two issues overwriting each other.	Stock is changed only by triggers T2 and T3 inside the same transaction as the issue/return, and guarded by `CHECK (available_qty BETWEEN 0 AND quantity)`.
6	Technical	In the stock trigger, the status had to reflect the new quantity, not the old one.	MySQL evaluates single-table `UPDATE … SET` assignments left to right, so `available_qty` is decremented first and the `IF(available_qty = 0, …)` that follows sees the new value.
7	Data handling	Loading sample data in the wrong order made trigger T1 reject historical bookings for facilities that were already marked under maintenance.	Fixed the load order: master data, then bookings, then equipment and issues, then maintenance.
8	Technical	Errors raised inside triggers (SIGNAL) were shown to users as raw stack traces at first.	The procedures use an EXIT HANDLER that rolls back and RESIGNALs; Flask catches `mysql.connector.Error` and shows `e.msg` as an alert.
9	Database design	Deleting cancelled bookings would lose history and break the usage statistics.	Bookings are never deleted; a `status` ENUM (CONFIRMED / CANCELLED / COMPLETED) is used, and reports filter on it.
10	Technical	Older MySQL versions (before 8.0.16) silently ignore CHECK constraints, which made one team member's tests pass when they should have failed.	All three of us standardized on MySQL 8.0 and added TC4–TC6 and TC10 to confirm the CHECKs are enforced.

Chapter 8: Conclusion and Future Enhancements

8.1 What Was Achieved

CampusHub meets all nine objectives set in the proposal. It replaces scattered registers with one MySQL database of 10 normalized tables that covers facilities, bookings, equipment,

issue/return and maintenance. Double booking is impossible: trigger T1 rejects overlapping slots and the locking transaction in `sp_create_booking` keeps this true under concurrent use. Equipment stock and facility status update themselves through triggers, and ten report queries and four views answer the questions that were hard to answer before, such as which facilities are free now, which items are overdue, and which departments use the facilities most. A Flask interface lets students, faculty and administrators use all of this from a browser.

8.2 Learning Outcomes

- Translating real requirements into an ER model, a relational schema and a 3NF design.
- Choosing data types, keys and constraints, and understanding what CHECK can and cannot enforce in MySQL.
- Writing triggers, stored procedures with error handlers, and views, and deciding which rules belong in the database rather than the application.
- Seeing ACID properties, isolation levels and row-level locking in action, including a real lost-update style race and its fix.
- Reading `EXPLAIN` plans and designing composite indexes for specific queries.
- Connecting a Python application to MySQL safely with pooled connections and parameterized queries.
- Working as a team of three with Git, dividing the database, backend and interface work.

8.3 Limitations

- There is no login or role-based access control in the web app; any user can open any page.
- Bookings must start and end on the same day; recurring bookings (for example every Monday for a semester) are not supported.
- Equipment is tracked by quantity, not by individual serial number, so a specific damaged unit cannot be identified.
- Booking status is not moved from CONFIRMED to COMPLETED automatically after the slot ends.
- No e-mail or SMS notifications for confirmations, overdue items or maintenance updates.
- The system runs on a local server and has been tested only with sample data, not at full campus scale.

8.4 Future Improvements

1. Authentication with roles (student, faculty, facility admin, department admin) and MySQL user privileges that match them (GRANT / REVOKE).
2. Recurring and multi-day bookings, plus an approval step for large halls.
3. A MySQL EVENT that marks past bookings COMPLETED and flags overdue equipment every night.
4. Serial-number tracking of equipment units and a damage/fine record on return.
5. E-mail notifications for confirmations, reminders and overdue items.
6. Deployment on a cloud database such as Amazon RDS for MySQL, with automated backups.
7. A calendar view of each facility and charts on the Reports page.
8. Integration with the university's ERP/VTOP login and student records.

8.5 References

1. Silberschatz, A., Korth, H. F. and Sudarshan, S., *Database System Concepts*, 7th edition, McGraw-Hill, 2019.
2. Elmasri, R. and Navathe, S. B., *Fundamentals of Database Systems*, 7th edition, Pearson, 2016.
3. Oracle Corporation, *MySQL 8.0 Reference Manual* – CHECK constraints, triggers, stored programs and InnoDB locking. https://dev.mysql.com/doc/refman/8.0/en/
4. Pallets Projects, *Flask Documentation*. https://flask.palletsprojects.com/

5. Oracle Corporation, *MySQL Connector/Python Developer Guide*. https://dev.mysql.com/doc/connector-python/en/